

Programmazione Orientata agli Oggetti

Sealed Types e
Java Record

Contenuti

- Definizione dei Tipi in Java
- Sealed Types (Java 17+)
 - Sigillatura di un tipo
 - Sintassi
 - Semantica
 - Applicazioni
 - Sealed ed Astrazione
 - Quando sigillare
 - Sealed vs Enum
- Java Record (Java 16+)
 - Introduzione
 - Sintassi
 - Semantica
- **`o instanceof <Type> t`**

Contenuti

- **Definizione dei Tipi in Java**
- Sealed Types (Java 17+)
 - Sigillatura di un tipo
 - Sintassi
 - Semantica
 - Applicazioni
 - Sealed ed Astrazione
 - Quando sigillare
 - Sealed vs Enum
- Java Record (Java 16+)
 - Introduzione
 - Sintassi
 - Semantica
- **`o instanceof <Type> t`**

Definizione dei Tipi in Java (1)

- Abbiamo già visto tutti i principali meccanismi per la definizione di tipo: classi, interface, classi astratte, ed anche costanti enumerative
- Ne esistono altri: l'insieme dei meccanismi offerti da Java si è arricchito nel susseguirsi delle versioni della piattaforma:

- Tipi enumerativi (<<) - Java 5+
- **Sealed Type** - Java 17+
- **Java Record** - Java 16+
- Classi nidificate (>>) - Java 1.1+

sicuramente tutti meno *fondanti* di quelli che abbiamo già visto

- ✓ La JVM «conosce» solo classi ed interface
- Ma contribuiscono a determinare l'espressività del sistema dei tipi in Java

Definizione dei Tipi in Java (2)

- L'insieme dei meccanismi per la modellazione dei tipi è un degli aspetti più caratterizzanti un linguaggio
- La loro espressività nel modellare un dominio di interesse è importante per determinare l'efficacia del linguaggio
- Su questi aspetti, oggettivamente, il dogma della retrocompatibilità ha portato la piattaforma Java ad “inseguire”
 - ✓ (o, forse, quasi ad “aspettare”...)
- Altri linguaggi di consistente diffusione hanno contribuito a selezionare i più promettenti ed utili meccanismi da adottare per la definizioni dei tipi
 - Soprattutto quelli nella «sfera» della programmazione funzionale
- ✓ La capacità di evolvere della piattaforma Java si è dimostrata importante, se non decisiva, per mantenere così a lungo i suoi attuali livelli di diffusione

Accoppiamento Intrinseco delle Implementazioni in una Gerarchia di Tipi (1)

- L'estensione di classi è uno dei principali meccanismi per la definizione di tipi offerti da Java
- Abbiamo avuto modo di capire come sia difficile progettare una classe senza già predisporla alla sue estensioni
- Si pensi al criterio di equivalenza, oppure all'ordinamento naturale per un'intera gerarchia di classi
- Esempio più evidente (ma non l'unico possibile):
 - ✓ Si finisce per progettare l'implementazione dei metodi `equals()`/`hashCode()`/`compareTo()` di una classe tenendo conto della loro implementazione in *tutte* le classi che compongono l'intera gerarchia dei tipi di appartenenza

Accoppiamento Intrinseco delle Implementazioni in una Gerarchia di Tipi (2)

- In alcuni casi le implementazioni delle classi di una medesima gerarchia di tipi finiscono per esibire un «intrinseco» livello di accoppiamento
- Impossibile da eliminare completamente
 - Diventa problematico cambiare il codice di un sottotipo concreto senza dover riconsiderare anche gli altri tipi nella gerarchia, od addirittura i tipi che *forse* si scriveranno in futuro
- ✓ Tanto vale renderli espliciti dicendo quali sono i sottotipi già quando si dichiara il supertipo?
- ✓ Tanto vale rinunciare a «astrarre» le differenze tra i sottotipi presenti ed accumunati dal supertipo comune?

«Sigillare» i Tipi

- L'atteggiamento della piattaforma Java rispetto all'estensione tra classi è stata, storicamente, un pochino “bipolare” (!)
 - Classi non **final**: chiunque e da dovunque può estendere una classe che gli risulta visibile ed accessibile
 - si usa dire «open for extension», ovvero, predisposto per l'estensione
 - Classi **final**: nessuno può estendere la classe sebbene sia accessibile
- A partire da Java 17+ (preview in Java 15) è stato introdotto un meccanismo per meglio controllare l'estensione dei tipi e «disaccoppiarla» dai meccanismi per la scelta della loro visibilità
- Alcune osservazioni alla base di questo ragionamento:
 - ✓ L'accessibilità e l'estensibilità possono seguire percorsi diversi
 - ✓ Non tutto ciò che è accessibile deve anche essere estendibile
 - ✓ L'accoppiamento intrinseco delle implementazioni delle classi di una medesima gerarchia talvolta motiva la definizione contestuale di tutte le implementazioni
- I tipi (classi, interface, e vedremo tra poco i Java «record») possono partecipare a gerarchie con tipi «sealed», lett. “*sigillati*”
 - i tipi marcati come **sealed** controllano esplicitamente l'insieme dei loro sottotipi che “permettono”: li elencano subito già in fase di loro definizione

Sintassi Tipi sealed

```
public sealed interface Forma permits Punto, Cerchio, Rettangolo, Gruppo {  
    public void trasla(int dx, int dy);  
}  
  
final class Cerchio implements Forma {  
    private int raggio;  
    private Punto centro;  
    //...  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}  
  
final class Rettangolo implements Forma {  
    //... ..  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}  
  
final class Gruppo implements Forma {  
    private List<Forma> componenti;  
    //... ..  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

Semantica Tipi sealed

- Un tipo sigillato può essere esteso/implementato solo dai sottotipi che ha elencato esplicitamente
 - **Forma** può essere implementata solo dai tipi che ha esplicitamente elencato
 - ✓ Se la clausola **permits** è omessa, si considerano tutti e soli i sottotipi nella stessa unità di compilazione come permessi
- Per ciascuno dei sottotipi elencati *dobbiamo* decidere l'esito della sigillatura per ciascun sottotipo:
 - **final**: non potrà più essere esteso
 - **sealed**: sigillerà, ricorsivamente, i suoi sottotipi
 - **non-sealed**: potrà essere esteso come sempre
- Senza una scelta esplicita non compila...

Sintassi Tipi sealed

```
public sealed interface Forma permits Punto, Cerchio, Rettangolo, Gruppo {  
    public void trasla(int dx, int dy);  
}
```

```
final class Cerchio implements Forma {  
    private int raggio;  
    private Punto centro;  
    //...  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

The class Gruppo with a sealed direct superclass or a sealed direct superinterface Forma should be declared either final, sealed, or non-sealed

```
final class Rettangolo implements Forma {  
    //... ..  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

```
final class Gruppo implements Forma {  
    private List<Forma> componenti;  
    //... ..  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

Sintassi non-sealed (1)

```
public sealed interface Forma permits Punto, Cerchio, Rettangolo, Gruppo {  
    public void trasla(int dx, int dy);  
}
```

```
final class Cerchio implements Forma {  
    private int raggio;  
    private Punto centro;  
    //...  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

non-sealed: Gruppo rimane libero di avere suoi sottotipi, anche in futuro, ad es. GruppoOrdinato e GruppoNonOrdinato

```
final class Rettangolo implements Forma {  
    //... ..  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

```
non-sealed class Gruppo implements Forma {  
    private List<Forma> componenti;  
    //... ..  
    @Override  
    public void trasla(int dx, int dy) { /* ... */ }  
}
```

Sintassi non-sealed (2)

```
import java.util.*;
public non-sealed class Gruppo implements Forma {
    @Override
    public void trasla(int dx, int dy) { /* ... */ }
}
final class GruppoOrdinato extends Gruppo {
    private List<Forma> componenti;
    //...
    @Override
    public void trasla(int dx, int dy) { /* ... */ }
}
final class GruppoNonOrdinato extends Gruppo {
    private Set<Forma> componenti;
    //...
    @Override
    public void trasla(int dx, int dy) { /* ... */ }
}
```

Applicazioni dei Tipi sealed

- L'autore di un tipo sigillato è libero di ragionare su tutte le possibili implementazioni perché le controlla
- Lo stesso vale anche per il compilatore, che può verificare la copertura di tutti i possibili sottotipi da parte di alcuni costrutti
 - Istruzioni di controllo **switch**
 - downcast
- Spesso i tipi sigillati si usano con i Java record (>>) Perché?
 - ✓ partono da una comune attitudine:
 - smettere di astrarre i sottotipi di una gerarchia e la loro implementazione dietro un supertipo comune che finisce per nascondere le specificità e l'implementazione

L'Astrazione di Tipo dal Punto di Vista dei Tipi Sigillati (1)

- Dal punto del p.d.s. i tipi sealed possono, addirittura, sorprendere...
- In generale, un supertipo serve a disaccoppiare gli utilizzatori dai suoi sottotipi, che cerca di «*nascondere*». Ad es.:
 - **Comando** disaccoppia **DiaDia** dai dettagli dei comandi implementati dai sottotipi concreti di **Comando**, come ad es. **ComandoVai**
 - ✓ Questo nonostante i comportamenti polimorfi, e quindi potenzialmente molto diversi, dei sottotipi presenti e futuri (... i *comandi* ancora "da inventare")
- In definitiva, un supertipo viene spesso ideato con l'obiettivo di lasciar utilizzare i suoi sottotipi indipendentemente dalle loro peculiarità
- L'incapsulamento ci ha permesso, talvolta, di non conoscere affatto il nome del tipo concreto utilizzato a tempo dinamico!
 - ✓ Es. **java.util.Iterator**

L'Astrazione di Tipo dal Punto di Vista dei Tipi Sigillati (2)

- Al contrario ai supertipi sigillati si richiede di «*esporre*» un elenco esaustivo, completo di nome completamente qualificato, di tutti i sottotipi presenti e futuri
 - a costo di non permettere altre estensioni, nemmeno future
- ✓ Esattamente l'opposto dell'astrazione (!)
 - A ben vedere, l'unico costruttore di tipo che abbiamo già visto e che finisce per esporre il nome di tutti e soli i suoi sottotipi sono le costanti enumerative
 - ✓ Partono dal presupposto si voglia conoscere il nome delle costanti, essendo poche, ben note, e di implem. scontata
- In quale situazioni può convenire la sigillatura di un tipo e perché?

Sigillatura di una Gerarchia

- Lasciar trapelare, al di là della “barriera astrattiva” creata dal supertipo comune, tutti i tipi di una gerarchia risulta utile in alcune situazioni
- Data una gerarchia di tipi, si può voler far arrivare agli utilizzatori
 - la conoscenza di tutti e soli i sottotipi che la compongono
 - la conoscenza delle loro implem. (magari perché banali)
- Il termine “sigillatura” rischia di essere leggermente fuorviante
 - Si «chiude» verso future estensioni dei possibili sottotipi
 - Ma allo stesso tempo si finisce per «aprire» agli utilizzatori la conoscenza di tutti e soli i tipi che compongono la gerarchia
 - come già accade per le costanti dei tipi enumerativi
- Quindi si «sigillando» una gerarchia “si espongono” definitivamente i tipi che la compongono

Quando Sigillare un Tipo?

- ✓ Ogni qualvolta l'astrazione che un tipo illimitatamente aperto all'estensione introduce risulta mal ripagata
- *Domini semplici e/o stabili*
 - i tipi di una gerarchia sono già noti e stabili nel tempo
 - Si pensi a tipi che modellano degli *standard* consolidati
 - Ad es.
 - La modellazione di codici di errore o di stato di un protocollo standard e consolidato, come ad es. *http*
 - La modellazione del risultato di una computazione asincrona (>>)
- *Astrazione difficile da motivare*
 - i livelli di indirizzione aggiuntivi sono motivo di confusione
 - gli utilizzatori finiscono quasi sempre per dover lavorare con i tipi concreti e quasi mai possono fermarsi al loro supertipo
 - l'apertura verso sottotipi futuri è difficile da predisporre a livello implementativo: meglio concentrarsi sul presente e “sigillare” i tipi già noti

Tipi Sealed vs Enum (1)

- Le costanti di un tipo enumerativo possono e devono avere uno stato immutabile

```
enum Planet { MERCURY, VENUS, EARTH, ... }
```

- Ogni costante enumerativa:
 - corrisponde ad un sottotipo distinto
 - possiede una sola ed unica istanza
- Pertanto è possibile associare dei dati ad ogni costante enumerativa

✓ ad es. massa e raggio del pianeta in questo esempio:

```
enum Planet {  
    MERCURY (3.303e+23, 2.4397e6),  
    VENUS (4.869e+24, 6.0518e6),  
    EARTH (5.976e+24, 6.37814e6),  
    ...  
}
```

- Tuttavia il costrutto di **enum** non permette di modellare *molteplici* istanze dello stesso tipo con uno stato immutabile

Tipi Sealed vs Enum (2)

- Per il seguente esempio è necessaria un **sealed** interface

```
sealed interface Celestial {  
    class Planet implements Celestial {  
        Planet(String name, double mass, double radius) { ... }  
    class Star implements Celestial {  
        Star(String name, double mass, double temperature) { ... }  
    class Comet implements Celestial {  
        Comet(String name, double period, LocalDateTime lastSeen) { ... }  
    }  
}
```

- Un tipo enumerativo *NON* è uno strumento di modellazione adatto a questo scenario
 - ✓ Bisogna essere liberi di creare molteplici *istanze* di ogni corpo celeste pur rimanendo capaci di elencare un elenco ben noto e stabile nel tempo di *generi* di corpi celesti

Tipi Sealed: Controesempio

- Un esempio in cui la sigillatura del tipo sarebbe assolutamente controproducente nonostante la semplicità del tipo? **`java.util.Iterator`**
 - Ogni nuovo tipo di collezione, presente e futura, deve poterlo liberamente reimplementare
 - E nessuno vuole conoscere, nemmeno il nome (!), del tipo dinamico utilizzato
- **`Iterator`** nasce come tipo aperto all'estensione
 - ideato per essere implementato da ogni tipo di collezione, presente e futura
 - La sua principale missione è proprio quella di permettere di elencare gli elementi della collezione sottostante in maniera uniforme come specificata da solo supertipo comune
 - nascondendo i dettagli delle sue implementazioni

Gerarchia di Tipi Semplici

- Data una gerarchia di tipi, si può voler esporre e far arrivare agli utilizzatori la conoscenza
 - sia di tutti e soli i sottotipi che la compongono
 - ✓ sia delle loro implementazioni (magari perché banali)
- Quando si decide di sigillare una gerarchia ed esporre tutti i sottotipi, lo si fa anche perché la gerarchia nel suo “complesso”, con tutte le allegate implementazioni dei sottotipi, risulta sufficientemente semplice
- La gerarchia, nel suo complesso, *NON* merita una modellazione con classi *illimitatamente* aperte all'estensione

Cerimoniale di `class`

- Da questo punto di vista, il costrutto di `class` ha obiettivi molto generali nella sua forma originale
 - aperta all'estensione *illimitata presente e futura*
 - supporta *qualsiasi* tipo di implementazione
- Finisce per esibire un «cerimoniale» che talvolta può risultare eccessivo per implementazioni banali e scontate
 - classi immutabili che si limitano ad aggregare un certo numero di proprietà
- Si rischia di *NON* introdurre alcuni tipi, che invece la modellazione di dominio richiederebbe, solo per la “pesantezza” del costrutto di `class`
- Arrivano in soccorso i Java *record*...

Tipi Sealed: Dettagli Finali

- La parola chiave **sealed** può essere applicata sia alle classi che alle interface
- Invece è un errore applicarla ad una classe od un tipo già finale o implicitamente tale come gli **enum** (ed i java **record**)
 - questi possono anche essere marcati **final** (senza provocare errori di compilazione) ma lo sono comunque, implicitamente
- Non ha senso, e non compila, una classe od una interface segnata con **non-sealed** se *non* possiede un supertipo diretto marcato come **sealed**

Contenuti

- Definizione dei Tipi in Java
- Sealed Types (Java 17+)
 - Sigillatura di un tipo
 - Sintassi
 - Semantica
 - Applicazioni
 - Sealed ed Astrazione
 - Quando sigillare
 - Sealed vs Enum
- **Java Record (Java 16+)**
 - Introduzione
 - Sintassi
 - Semantica
- **`o instanceof <Type> t`**

Java Record: Introduzione

- Introdotti in Java 14 e rilasciati in Java 16
- Permettono di modellare oggetti immutabili
 - di implementazione “scontata”
 - completamente specificati da un elenco di proprietà che ospitano
- Da questa lista di proprietà risultano naturalmente determinati
 - il costruttore delle istanze del tipo
 - i metodi accessori alle proprietà
 - il criterio di equivalenza
 - metodi **`equals()`** ed **`hashCode()`**
 - il metodo **`toString()`**
- Tutti i record definiti hanno come supertipo comune **`java.lang.Record`**
 - ✓ con un meccanismo simile agli **`enum`**, che ereditano da **`java.lang.Enum`**
- Proprio per mantenere semplice ed auto-contenuta l’implementazione, i **`java record`**, come anche gli **`enum`**, vengono implicitamente considerati **`final`** e non possono essere estesi ne dichiarati **`abstract`**

Esempio: Un'Intera Gerarchia con i Java Record

```
import java.time.LocalDateTime;
```

```
sealed interface Celestial {
```

```
    record Planet(String name,  
                  double mass,  
                  double radius)
```

```
        implements Celestial {}
```

```
    record Star(String name,  
                double mass,  
                double temperature)
```

```
        implements Celestial {}
```

```
    record Comet(String name,  
                 double period,  
                 LocalDateTime lastSeen)
```

```
        implements Celestial {}
```

```
}
```

Elenco di proprietà

Specificate direttamente dopo il nome di tipo record

Java Record: Motivazioni

- È facile definire un tipo immutabile del tutto equivalente utilizzando il classico costrutto di classe
 - ✓ Basta utilizzare una variabile **final private** per ogni proprietà ...e quindi scrivere costruttori, ed altri metodi di conseguenza
- Ma la sintassi con i Java **record** risulta fortemente snellita
- Può essere un errore concentrarsi solo ed esclusivamente sulla riduzione di verbosità che il costrutto assicura rispetto al più generale costrutto di **class**
- Il vero punto, forse, è che, senza il cerimoniale delle classi, si ha un motivo in meno per omettere la definizione di alcuni tipi
- Il costrutto di **record**, come anche i tipi **sealed**, si utilizzano quando si ha l'intenzione di esporre i sottotipi di una gerarchia
 - Per questo motivo i due costrutti spesso si usano assieme
 - ✓ Ma **sealed** si concentra sull'estendibilità, **record** sull'implementazione

Java Record: Sintassi

- Riprendiamo l'esempio delle forme geometriche ma cambiamo la segnatura del metodo `trasla()` per modellare forme che ora vogliamo *immutabili* una volta create. La segnatura del metodo `trasla()` cambia:

```
public sealed interface Forma
    permits Punto, Cerchio, Rettangolo, Gruppo {
    public Forma trasla(int dx, int dy);
}
```

- Riconsideriamo il tipo `Punto`

```
public record Punto(int x, int y)
    implements Forma {
    @Override
    public Punto trasla(int dx, int dy) {
        return new Punto(x()+dx, y()+dy());
    }
}
```

Elenco di proprietà

- Le proprietà sono conservate (nel `.class` generate) da variabili di istanza `final`

Forme con Record: Esempio Completo (di Implementazioni)

```
public sealed interface Forma
    permits Punto, Cerchio, Rettangolo {
    public Forma trasla(int dx, int dy);
}

final record Cerchio(Punto centro, int raggio)
    implements Forma {
    @Override
    public Cerchio trasla(int dx, int dy)
    { return new Cerchio(centro.trasla(dx, dy), raggio()); }
}

final record Rettangolo(Punto vertice, int altezza, int base)
    implements Forma {
    @Override
    public Rettangolo trasla(int dx, int dy)
    { return new Rettangolo(vertice.trasla(dx, dy), altezza(), base()); }
}
```

Java Record: Semantica

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class PuntoTest {

    @Test
    public void testJavaRecord() {
        assertEquals(new Punto(2,3), new Punto(2,3));
        assertNotSame(new Punto(2,3), new Punto(2,3));
        assertEquals(2, new Punto(2,3).x());
        assertEquals(3, new Punto(2,3).y());
        assertEquals("Punto[x=2, y=3]", new Punto(2,3).toString());
    }
}
```

- I record si istanziano con la sintassi usuale: **new Punto(2,3)**
 - ✓ il costruttore è implicitamente definito sulla base delle proprietà
- Il criterio di equivalenza considera tutte le proprietà e non è necessario preoccuparsi di utilizzare **==** per variabili di istanza primitive oppure **equals()** su quelle che sono riferimenti ad oggetti

Java Record: Metodi Accessori

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class PuntoTest {

    @Test
    public void testJavaRecord() {
        assertEquals(new Punto(2,3), new Punto(2,3));
        assertNotSame(new Punto(2,3), new Punto(2,3));
        assertEquals(2, new Punto(2,3).x());
        assertEquals(3, new Punto(2,3).y());
        assertEquals("Punto[x=2, y=3]", new Punto(2,3).toString());
    }
}
```

- I metodi accessori prendono il nome della proprietà (ad es. `.x()`): non hanno i prefissi tipici dei metodi *getter*
 - ✓ come in `int getX()` od in `boolean isDegenerare()`

Java Record: Metodo toString()

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class PuntoTest {

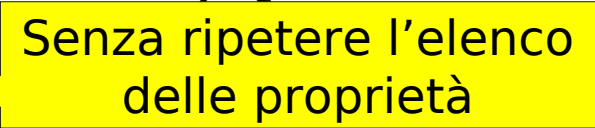
    @Test
    public void testJavaRecord() {
        assertEquals(new Punto(2,3), new Punto(2,3));
        assertNotSame(new Punto(2,3), new Punto(2,3));
        assertEquals(2, new Punto(2,3).x());
        assertEquals(3, new Punto(2,3).y());
        assertEquals("Punto[x=2, y=3]", new Punto(2,3).toString());
    }
}
```

- Il metodo **toString()** viene automaticamente definito per permettere stampe di record in cui al nome non completamente qualificato del tipo segue l'elenco ordinato delle proprietà con il corrispondente nome e valore

Java Record: Costruttori Espliciti

- È possibile anche definire esplicitamente costruttori
- Ad esempio per inserire dei controlli sulla validità dei parametri ricevuti

```
public record Range(int lo, int hi) {  
    public Range {  
        if (lo > hi)  
            throw new IllegalArgumentException();  
    }  
}
```



Senza ripetere l'elenco delle proprietà

- È possibile anche aggiungere altri metodi, anche statici
- ✓ *NON* si possono invece dichiarare altre variabili di istanza
- Attenzione: non ha senso complicare oltremodo la definizione di un Java record con metodi aggiuntivi
- Si finisce per contraddire l'originale intento del costrutto: essere una implementazione banale e scontatamente costruita "attorno" ad un certo numero di proprietà
- Può avere senso, invece, e fa spesso comodo, dichiarare qualche metodo statico, ad esempio metodi factory

Contenuti

- Definizione dei Tipi in Java
- Sealed Types (Java 17+)
 - Sigillatura di un tipo
 - Sintassi
 - Semantica
 - Applicazioni
 - Sealed ed Astrazione
 - Quando sigillare
 - Sealed vs Enum
- Java Record (Java 16+)
 - Introduzione
 - Sintassi
 - Semantica
- **o instanceof <Type> t**

obj instanceof <Type> t

- Sia il costrutto di **record**, sia i tipi **sealed**, espongono i sottotipi di una gerarchia
 - Gli utilizzatori finiscono spesso per lavorare direttamente con i sottotipi
 - E possono dover far frequenti, ed ineleganti, *downcast*
- Questo (ed altro) ha motivato l'introduzione (Java 16+) di una variante dell'operatore **instanceof** che evita la tradizionale sintassi del downcast.
 - ✓ N.B. a tempo di esecuzione, il downcast rimane perfettamente là dove già era!

- Ad esempio: **o instanceof String s**

In sostanza si accorpano:

- l'operatore **instanceof** nella forma già nota
- il downcast verso uno specificato sottotipo
- la dichiarazione di una variabile locale di tipo statico utile

- Esempio di utilizzo in un metodo:

```
class Calcolatore {  
    public static double calcolaArea(Forma f) {  
        if (f instanceof Cerchio c) {  
            return Math.PI * c.raggio() * c.raggio();  
        }  
        ...  
    }  
}
```

Conclusioni

- La piattaforma Java include un linguaggio *general-purpose* tuttora oggetto di revisioni significative per costrutti così fondamentali come quelli utilizzati per la definizione dei tipi
- Il futuro della piattaforma dipende dalla capacità di innovarsi e di inglobare le migliorie più convincenti
 - anche se proposte e sperimentate in altri linguaggi meno diffusi
 - ✓ l'importante è rendere non così appetibile il cambio di linguaggio
- Questo nonostante il *dogma* della retrocompatibilità
- Per ambiti più verticali, continueranno a nascere linguaggi concorrenti migliori, ma più specializzati
- Uno dei successori di Java con più trazione è *kotlin*,
 - sembra aver trovato un suo spazio convincendo una porzione di programmatori Java a migrare, senza “spaventarli”
 - al momento su questo specifico aspetto, altri linguaggi, come *scala* sembrano aver già fallito e difficilmente possono cambiare la situazione nei prossimi anni

Riferimenti

- Articoli di Brian Goetz, il responsabile delle proposte di queste estensioni alla piattaforma Java
 - <https://www.infoq.com/articles/java-sealed-classes/>
 - <https://www.infoq.com/articles/java-14-feature-spotlight/>
- JEP (Java Enhancement Proposal)
 - <https://openjdk.org/jeps/394>
 - <https://openjdk.org/jeps/395>
 - <https://openjdk.org/jeps/409>